Get 50% off your ticket to MongoDB.local London on October 2. Use code **Web50** READ MORE

MongoDB Products Resources Solutions Company Pricing Eng Support Sign In Try Free

Docs Home
MongoDB Manual
7.0 (current)
Introduction
Installation
MongoDB Shell (mongosh)
CRUD Operations
Aggregation Operations
Aggregation Pipeline
Reference
Commands
Stages
Operators
$abs
$accumulator
$acos
$acosh

Docs Home / MongoDB Manual / Aggregation Operations / Reference

On this page
Compatibility
Expression Operators
Arithmetic Expression Operators
Array Expression Operators
Bitwise Operators
Boolean Expression Operators
Comparison Expression Operators
Conditional Expression Operators
Custom Aggregation Expression Operators
Data Size Operators
Date Expression Operators
Literal Expression Operators

Aggregation Operators

The aggregation pipeline operators are compatible with MongoDB Atlas and on-premise environments.

NOTE

For details on a specific operator, including syntax and examples, click on the link to the operator's reference page.

Compatibility

You can use the aggregation pipeline operators for deployments hosted in the following environments:

- MongoDB Atlas: The fully managed service for MongoDB deployments in the cloud
- MongoDB Enterprise: The subscription-based, self-managed version of MongoDB
- MongoDB Community: The source-available, free-to-use, and self-managed version of MongoDB

Expression Operators

Expression operators are similar to functions that take arguments. In general, these operators take an array of arguments and have the following form:

```
{ <operator>: [ <argument1>, <argument2> ... ] }
```

If an operator accepts a single argument, you can omit the outer array designating the argument list:

```
{ <operator>: <argument> }
```

To avoid parsing ambiguity if the argument is a literal array, you must wrap the literal array in a `$literal` expression or keep the outer array that designates the argument list.

This page lists the available operators that can be used to construct expressions. Expressions are used in the following contexts in MQL:

- Some aggregation pipeline stages, such as `$project`, `$addFields`, and `$group`
- Inside query predicates that use `$expr`
- Find query projections

Arithmetic Expression Operators

Arithmetic expressions perform mathematic operations on numbers. Some arithmetic expressions can also support date arithmetic.

Name	Description
`$abs`	Returns the absolute value of a number.
`$add`	Adds numbers to return the sum, or adds numbers and a date to return a new date. If adding numbers and a date, treats the numbers as milliseconds. Accepts any number of argument expressions, but at most, one expression can resolve to a date.
`$ceil`	Returns the smallest integer greater than or equal to the specified number.
`$divide`	Returns the result of dividing the first number by the second. Accepts two argument expressions.
`$exp`	Raises *e* to the specified exponent.
`$floor`	Returns the largest integer less than or equal to the specified number.
`$ln`	Calculates the natural log of a number.
`$log`	Calculates the log of a number in the specified base.
`$log10`	Calculates the log base 10 of a number.
`$mod`	Returns the remainder of the first number divided by the second. Accepts two argument expressions.
`$multiply`	Multiplies numbers to return the product. Accepts any number of argument expressions.
`$pow`	Raises a number to the specified exponent.
`$round`	Rounds a number to to a whole integer or to a specified decimal place.
`$sqrt`	Calculates the square root.
`$subtract`	Returns the result of subtracting the second value from the first. If the two values are numbers, return the difference. If the two values are dates, return the difference in milliseconds. If the two values are a date and a number in milliseconds, return the resulting date. Accepts two argument expressions. If the two values are a date and a number, specify the date argument first as it is not meaningful to subtract a date from a number.
`$trunc`	Truncates a number to a whole integer or to a specified decimal place.

Array Expression Operators

Name	Description
`$arrayElemAt`	Returns the element at the specified array index.
`$arrayToObject`	Converts an array of key value pairs to a document.
`$concatArrays`	Concatenates arrays to return the concatenated array.
`$filter`	Selects a subset of the array to return an array with only the elements that match the filter condition.
`$firstN`	Returns a specified number of elements from the beginning of an array. Distinct from the `$firstN` accumulator.
`$in`	Returns a boolean indicating whether a specified value is in an array.
`$indexOfArray`	Searches an array for an occurrence of a specified value and returns the array index of the first occurrence. Array indexes start at zero.
`$isArray`	Determines if the operand is an array. Returns a boolean.
`$lastN`	Returns a specified number of elements from the end of an array. Distinct from the `$lastN` accumulator.
`$map`	Applies a subexpression to each element of an array and returns the array of resulting values in order. Accepts named parameters.
`$maxN`	Returns the `n` largest values in an array. Distinct from the `$maxN` accumulator.
`$minN`	Returns the `n` smallest values in an array. Distinct from the `$minN` accumulator.
`$objectToArray`	Converts a document to an array of documents representing key-value pairs.
`$range`	Outputs an array containing a sequence of integers according to user-defined inputs.
`$reduce`	Applies an expression to each element in an array and combines them into a single value.
`$reverseArray`	Returns an array with the elements in reverse order.
`$size`	Returns the number of elements in the array. Accepts a single expression as argument.
`$slice`	Returns a subset of an array.
`$sortArray`	Sorts the elements of an array.
`$zip`	Merge two arrays together.

Bitwise Operators

Name	Description
`$bitAnd`	Returns the result of a bitwise `and` operation on an array of `int` or `long` values. *New in version 6.3.*
`$bitNot`	Returns the result of a bitwise `not` operation on a single argument or an array that contains a single `int` or `long` value. *New in version 6.3.*
`$bitOr`	Returns the result of a bitwise `or` operation on an array of `int` or `long` values. *New in version 6.3.*
`$bitXor`	Returns the result of a bitwise `xor` (exclusive or) operation on an array of `int` and `long` values. *New in version 6.3.*

Boolean Expression Operators

Boolean expressions evaluate their argument expressions as booleans and return a boolean as the result.

In addition to the `false` boolean value, Boolean expression evaluates as `false` the following: `null`, `0`, and `undefined` values. The Boolean expression evaluates all other values as `true`, including non-zero numeric values and arrays.

Name	Description
`$and`	Returns `true` only when *all* its expressions evaluate to `true`. Accepts any number of argument expressions.
`$not`	Returns the boolean value that is the opposite of its argument expression. Accepts a single argument expression.
`$or`	Returns `true` when *any* of its expressions evaluates to `true`. Accepts any number of argument expressions.

Comparison Expression Operators

Comparison expressions return a boolean except for `$cmp` which returns a number.

The comparison expressions take two argument expressions and compare both value and type, using the specified BSON comparison order for values of different types.

Name	Description
`$cmp`	Returns `0` if the two values are equivalent, `1` if the first value is greater than the second, and `-1` if the first value is less than the second.
`$eq`	Returns `true` if the values are equivalent.
`$gt`	Returns `true` if the first value is greater than the second.
`$gte`	Returns `true` if the first value is greater than or equal to the second.
`$lt`	Returns `true` if the first value is less than the second.
`$lte`	Returns `true` if the first value is less than or equal to the second.
`$ne`	Returns `true` if the values are *not* equivalent.

Conditional Expression Operators

Name	Description
`$cond`	A ternary operator that evaluates one expression, and depending on the result, returns the value of one of the other two expressions. Accepts either three expressions in an ordered list or three named parameters.
`$ifNull`	Returns either the non-null result of the first expression or the result of the second expression if the first expression results in a null result. Null result encompasses instances of undefined values or missing fields. Accepts two expressions as arguments. The result of the second expression can be null.
`$switch`	Evaluates a series of case expressions. When it finds an expression which evaluates to `true`, `$switch` executes a specified expression and breaks out of the control flow.

Custom Aggregation Expression Operators

Name	Description
`$accumulator`	Defines a custom accumulator function.
`$function`	Defines a custom function.

Data Size Operators

The following operators return the size of a data element:

Name	Description
`$binarySize`	Returns the size of a given string or binary data value's content in bytes.
`$bsonSize`	Returns the size in bytes of a given document (i.e. bsontype `Object`) when encoded as BSON.

Date Expression Operators

The following operators returns date objects or components of a date object:

Name	Description
`$dateAdd`	Adds a number of time units to a date object.
`$dateDiff`	Returns the difference between two dates.
`$dateFromParts`	Constructs a BSON Date object given the date's constituent parts.
`$dateFromString`	Converts a date/time string to a date object.
`$dateSubtract`	Subtracts a number of time units from a date object.
`$dateToParts`	Returns a document containing the constituent parts of a date.
`$dateToString`	Returns the date as a formatted string.
`$dateTrunc`	Truncates a date.
`$dayOfMonth`	Returns the day of the month for a date as a number between 1 and 31.
`$dayOfWeek`	Returns the day of the week for a date as a number between 1 (Sunday) and 7 (Saturday).
`$dayOfYear`	Returns the day of the year for a date as a number between 1 and 366 (leap year).
`$hour`	Returns the hour for a date as a number between 0 and 23.
`$isoDayOfWeek`	Returns the weekday number in ISO 8601 format, ranging from `1` (for Monday) to `7` (for Sunday).
`$isoWeek`	Returns the week number in ISO 8601 format, ranging from `1` to `53`. Week numbers start at `1` with the week (Monday through Sunday) that contains the year's first Thursday.
`$isoWeekYear`	Returns the year number in ISO 8601 format. The year starts with the Monday of week 1 (ISO 8601) and ends with the Sunday of the last week (ISO 8601).
`$millisecond`	Returns the milliseconds of a date as a number between 0 and 999.
`$minute`	Returns the minute for a date as a number between 0 and 59.
`$month`	Returns the month for a date as a number between 1 (January) and 12 (December).
`$second`	Returns the seconds for a date as a number between 0 and 60 (leap seconds).
`$toDate`	Converts value to a Date.
`$week`	Returns the week number for a date as a number between 0 (the partial week that precedes the first Sunday of the year) and 53 (leap year).
`$year`	Returns the year for a date as a number (e.g. 2014).

The following arithmetic operators can take date operands:

Name	Description
`$add`	Adds numbers and a date to return a new date. If adding numbers and a date, treats the numbers as milliseconds. Accepts any number of argument expressions, but at most, one expression can resolve to a date.
`$subtract`	Returns the result of subtracting the second value from the first. If the two values are dates, return the difference in milliseconds. If the two values are a date and a number in milliseconds, return the resulting date. Accepts two argument expressions. If the two values are a date and a number, specify the date argument first as it is not meaningful to subtract a date from a number.

Literal Expression Operator

Name	Description
`$literal`	Return a value without parsing. Use for values that the aggregation pipeline may interpret as an expression. For example, use a `$literal` expression to a string that starts with a dollar sign (`$`) to avoid parsing as a field path.

Miscellaneous Operators

Name	Description
`$getField`	Returns the value of a specified field from a document. You can use `$getField` to retrieve the value of fields with names that contain periods (`.`) or start with dollar signs (`$`). *New in version 5.0.*
`$rand`	Returns a random float between 0 and 1
`$sampleRate`	Randomly select documents at a given rate. Although the exact number of documents selected varies on each run, the quantity chosen approximates the sample rate expressed as a percentage of the total number of documents.
`$toHashedIndexKey`	Computes and returns the hash of the input expression using the same hash function that MongoDB uses to create a hashed index.

Object Expression Operators

Name	Description
`$mergeObjects`	Combines multiple documents into a single document.
`$objectToArray`	Converts a document to an array of documents representing key-value pairs.
`$setField`	Adds, updates, or removes a specified field in a document. You can use `$setField` to add, update, or remove fields with names that contain periods (`.`) or start with dollar signs (`$`). *New in version 5.0.*

Set Expression Operators

Set expressions performs set operation on arrays, treating arrays as sets. Set expressions ignores the duplicate entries in each input array and the order of the elements.

If the set operation returns a set, the operation filters out duplicates in the result to output an array that contains only unique entries. The order of the elements in the output array is unspecified.

If a set contains a nested array element, the set expression does *not* descend into the nested array but evaluates the array at top-level.

Name	Description
`$allElementsTrue`	Returns `true` if *no* element of a set evaluates to `false`, otherwise, returns `false`. Accepts a single argument expression.
`$anyElementTrue`	Returns `true` if any elements of a set evaluate to `true`; otherwise, returns `false`. Accepts a single argument expression.
`$setDifference`	Returns a set with elements that appear in the first set but not in the second set; i.e. performs a relative complement of the second set relative to the first. Accepts exactly two argument expressions.
`$setEquals`	Returns `true` if the input sets have the same distinct elements. Accepts two or more argument expressions.
`$setIntersection`	Returns a set with elements that appear in *all* of the input sets. Accepts any number of argument expressions.
`$setIsSubset`	Returns `true` if all elements of the first set appear in the second set, including when the first set equals the second set; i.e. not a strict subset. Accepts exactly two argument expressions.
`$setUnion`	Returns a set with elements that appear in *any* of the input sets.

String Expression Operators

String expressions, with the exception of `$concat`, only have a well-defined behavior for strings of ASCII characters.

`$concat` behavior is well-defined regardless of the characters used.

Name	Description
`$concat`	Concatenates any number of strings.
`$dateFromString`	Converts a date/time string to a date object.
`$dateToString`	Returns the date as a formatted string.
`$indexOfBytes`	Searches a string for an occurrence of a substring and returns the UTF-8 byte index of the first occurrence. If the substring is not found, returns `-1`.
`$indexOfCP`	Searches a string for an occurrence of a substring and returns the UTF-8 code point index of the first occurrence. If the substring is not found, returns `-1`
`$ltrim`	Removes whitespace or the specified characters from the beginning of a string.
`$regexFind`	Applies a regular expression (regex) to a string and returns information on the *first* matched substring.
`$regexFindAll`	Applies a regular expression (regex) to a string and returns information on the all matched substrings.
`$regexMatch`	Applies a regular expression (regex) to a string and returns a boolean that indicates if a match is found or not.
`$replaceOne`	Replaces the first instance of a matched string in a given input.
`$replaceAll`	Replaces all instances of a matched string in a given input.
`$rtrim`	Removes whitespace or the specified characters from the end of a string.
`$split`	Splits a string into substrings based on a delimiter. Returns an array of substrings. If the delimiter is not found within the string, returns an array containing the original string.
`$strLenBytes`	Returns the number of UTF-8 encoded bytes in a string.
`$strLenCP`	Returns the number of UTF-8 code points in a string.
`$strcasecmp`	Performs case-insensitive string comparison and returns: `0` if two strings are equivalent, `1` if the first string is greater than the second, and `-1` if the first string is less than the second.
`$substr`	Deprecated. Use `$substrBytes` or `$substrCP`.
`$substrBytes`	Returns the substring of a string. Starts with the character at the specified UTF-8 byte index (zero-based) in the string and continues for the specified number of bytes.
`$substrCP`	Returns the substring of a string. Starts with the character at the specified UTF-8 code point (CP) index (zero-based) in the string and continues for the number of code points specified.
`$toLower`	Converts a string to lowercase. Accepts a single argument expression.
`$toString`	Converts value to a string.
`$trim`	Removes whitespace or the specified characters from the beginning and end of a string.
`$toUpper`	Converts a string to uppercase. Accepts a single argument expression.

Text Expression Operator

Name	Description
`$meta`	Access available per-document metadata related to the aggregation operation.

Timestamp Expression Operators

Timestamp expression operators return values from a timestamp.

Name	Description
`$tsIncrement`	Returns the incrementing ordinal from a timestamp as a `long`. *New in version 5.1.*
`$tsSecond`	Returns the seconds from a timestamp as a `long`. *New in version 5.1.*

Trigonometry Expression Operators

Trigonometry expressions perform trigonometric operations on numbers. Values that represent angles are always input or output in radians. Use `$degreesToRadians` and `$radiansToDegrees` to convert between degree and radian measurements.

Name	Description
`$sin`	Returns the sine of a value that is measured in radians.
`$cos`	Returns the cosine of a value that is measured in radians.
`$tan`	Returns the tangent of a value that is measured in radians.
`$asin`	Returns the inverse sin (arc sine) of a value in radians.
`$acos`	Returns the inverse cosine (arc cosine) of a value in radians.
`$atan`	Returns the inverse tangent (arc tangent) of a value in radians.
`$atan2`	Returns the inverse tangent (arc tangent) of `y / x` in radians, where `y` and `x` are the first and second values passed to the expression respectively.
`$asinh`	Returns the inverse hyperbolic sine (hyperbolic arc sine) of a value in radians.
`$acosh`	Returns the inverse hyperbolic cosine (hyperbolic arc cosine) of a value in radians.
`$atanh`	Returns the inverse hyperbolic tangent (hyperbolic arc tangent) of a value in radians.
`$sinh`	Returns the hyperbolic sine of a value that is measured in radians.
`$cosh`	Returns the hyperbolic cosine of a value that is measured in radians.
`$tanh`	Returns the hyperbolic tangent of a value that is measured in radians.
`$degreesToRadians`	Converts a value from degrees to radians.
`$radiansToDegrees`	Converts a value from radians to degrees.

Type Expression Operators

Name	Description
`$convert`	Converts a value to a specified type.
`$isNumber`	Returns boolean `true` if the specified expression resolves to an `integer`, `decimal`, `double`, or `long`. Returns boolean `false` if the expression resolves to any other BSON type, `null`, or a missing field.
`$toBool`	Converts value to a boolean.
`$toDate`	Converts value to a Date.
`$toDecimal`	Converts value to a Decimal128.
`$toDouble`	Converts value to a double.
`$toInt`	Converts value to an integer.
`$toLong`	Converts value to a long.
`$toObjectId`	Converts value to an ObjectId.
`$toString`	Converts value to a string.
`$type`	Return the BSON data type of the field.

Accumulators (`$group, $bucket, $bucketAuto, $setWindowFields`)

Aggregation accumulator operators:

- Maintain their state as documents progress through the aggregation pipeline.
- Return totals, maxima, minima, and other values.
- Can be used in these aggregation pipeline stages:
  - `$bucket`
  - `$bucketAuto`
  - `$group`
  - `$setWindowFields` starting in MongoDB 5.0 (except when you are using the `$accumulator` or `$mergeObjects` operators, which cannot be used with `$setWindowFields`)

Changed in version 5.0.

Name	Description
`$accumulator`	Returns the result of a user-defined accumulator function.
`$addToSet`	Returns an array of *unique* expression values for each group. Order of the array elements is undefined. *Changed in version 5.0*: Available in the `$setWindowFields` stage.
`$avg`	Returns an average of numerical values. Ignores non-numeric values. *Changed in version 5.0*: Available in the `$setWindowFields` stage.
`$bottom`	Returns the bottom element within a group according to the specified sort order. *New in version 5.2.* Available in the `$group` and `$setWindowFields` stages.
`$bottomN`	Returns an aggregation of the bottom `n` fields within a group, according to the specified sort order. *New in version 5.2.* Available in the `$group` and `$setWindowFields` stages.
`$count`	Returns the number of documents in a group. Distinct from the `$count` pipeline stage. *New in version 5.0*: Available in the `$group` and `$setWindowFields` stages.
`$first`	Returns the result of an expression for the first document in a group. *Changed in version 5.0*: Available in the `$setWindowFields` stage.
`$firstN`	Returns an aggregation of the first `n` elements within a group. Only meaningful when documents are in a defined order. Distinct from the `$firstN` array operator. *New in version 5.2*: Available in the `$group`, expression and `$setWindowFields` stages.
`$last`	Returns the result of an expression for the last document in a group. *Changed in version 5.0*: Available in the `$setWindowFields` stage.
`$lastN`	Returns an aggregation of the last `n` elements within a group. Only meaningful when documents are in a defined order. Distinct from the `$lastN` array operator. *New in version 5.2*: Available in the `$group`, expression and `$setWindowFields` stages.
`$max`	Returns the highest expression value for each group. *Changed in version 5.0*: Available in the `$setWindowFields` stage.
`$maxN`	Returns an aggregation of the `n` maximum valued elements in a group. Distinct from the `$maxN` array operator. *New in version 5.2.* Available in `$group`, `$setWindowFields` and as an expression.
`$median`	Returns an approximation of the median, the 50th percentile, as a scalar value. *New in version 7.0.* This operator is available as an accumulator in these stages: • `$group` • `$setWindowFields` It is also available as an aggregation expression.
`$mergeObjects`	Returns a document created by combining the input documents for each group.
`$min`	Returns the lowest expression value for each group. *Changed in version 5.0*: Available in the `$setWindowFields` stage.
`$minN`	Returns an aggregation of the `n` minimum valued elements in a group. Distinct from the `$minN` array operator. *New in version 5.2.* Available in `$group`, `$setWindowFields` and as an expression.
`$percentile`	Returns an array of scalar values that correspond to specified percentile values. *New in version 7.0.* This operator is available as an accumulator in these stages: • `$group` • `$setWindowFields` It is also available as an aggregation expression.
`$push`	Returns an array of expression values for documents in each group. *Changed in version 5.0*: Available in the `$setWindowFields` stage.
`$stdDevPop`	Returns the population standard deviation of the input values. *Changed in version 5.0*: Available in the `$setWindowFields` stage.

`$stdDevSamp`	Returns the sample standard deviation of the input values. *Changed in version 5.0*: Available in the `$setWindowFields` stage.
`$sum`	Returns a sum of numerical values. Ignores non-numeric values. *Changed in version 5.0*: Available in the `$setWindowFields` stage.
`$top`	Returns the top element within a group according to the specified sort order. *New in version 5.2.* Available in the `$group` and `$setWindowFields` stages.
`$topN`	Returns an aggregation of the top `n` fields within a group, according to the specified sort order. *New in version 5.2.* Available in the `$group` and `$setWindowFields` stages.

Accumulators (in Other Stages)

Some operators that are available as accumulators for the `$group` stage are also available for use in other stages but not as accumulators. When used in these other stages, these operators do not maintain their state and can take as input either a single argument or multiple arguments. For details, refer to the specific operator page.

Changed in version 5.0.

The following accumulator operators are also available in the `$project`, `$addFields`, `$set`, and, starting in MongoDB 5.0, the `$setWindowFields` stages.

Name	Description
`$avg`	Returns an average of the specified expression or list of expressions for each document. Ignores non-numeric values.
`$first`	Returns the result of an expression for the first document in a group.
`$last`	Returns the result of an expression for the last document in a group.
`$max`	Returns the maximum of the specified expression or list of expressions for each document
`$median`	Returns an approximation of the median, the 50th percentile, as a scalar value. *New in version 7.0.* This operator is available as an accumulator in these stages: • `$group` • `$setWindowFields` It is also available as an aggregation expression.
`$min`	Returns the minimum of the specified expression or list of expressions for each document
`$percentile`	Returns an array of scalar values that correspond to specified percentile values. *New in version 7.0.* This operator is available as an accumulator in these stages: • `$group` • `$setWindowFields` It is also available as an aggregation expression.
`$stdDevPop`	Returns the population standard deviation of the input values.
`$stdDevSamp`	Returns the sample standard deviation of the input values.
`$sum`	Returns a sum of numerical values. Ignores non-numeric values.

Variable Expression Operators

Name	Description
`$let`	Defines variables for use within the scope of a subexpression and returns the result of the subexpression. Accepts named parameters. Accepts any number of argument expressions.

Window Operators

New in version 5.0.

Window operators return values from a defined span of documents from a collection, known as a *window*. A window is defined in the `$setWindowFields` stage, available starting in MongoDB 5.0.

The following window operators are available in the `$setWindowFields` stage.

Name	Description
`$addToSet`	Returns an array of all unique values that results from applying an expression to each document. *Changed in version 5.0*: Available in the `$setWindowFields` stage.
`$avg`	Returns the average for the specified expression. Ignores non-numeric values. *Changed in version 5.0*: Available in the `$setWindowFields` stage.
`$bottom`	Returns the bottom element within a group according to the specified sort order. *New in version 5.2.* Available in the `$group` and `$setWindowFields` stages.
`$bottomN`	Returns an aggregation of the bottom `n` fields within a group, according to the specified sort order. *New in version 5.2.* Available in the `$group` and `$setWindowFields` stages.
`$count`	Returns the number of documents in the group or window. Distinct from the `$count` pipeline stage. *New in version 5.0.*
`$covariancePop`	Returns the population covariance of two numeric expressions. *New in version 5.0.*
`$covarianceSamp`	Returns the sample covariance of two numeric expressions. *New in version 5.0.*
`$denseRank`	Returns the document position (known as the rank) relative to other documents in the `$setWindowFields` stage partition. There are no gaps in the ranks. Ties receive the same rank. *New in version 5.0.*
`$derivative`	Returns the average rate of change within the specified window. *New in version 5.0.*
`$documentNumber`	Returns the position of a document (known as the document number) in the `$setWindowFields` stage partition. Ties result in different adjacent document numbers. *New in version 5.0.*
`$expMovingAvg`	Returns the exponential moving average for the numeric expression. *New in version 5.0.*
`$first`	Returns the result of an expression for the first document in a group or window. *Changed in version 5.0*: Available in the `$setWindowFields` stage.
`$integral`	Returns the approximation of the area under a curve. *New in version 5.0.*
`$last`	Returns the result of an expression for the last document in a group or window. *Changed in version 5.0*: Available in the `$setWindowFields` stage.
`$linearFill`	Fills `null` and missing fields in a window using linear interpolation based on surrounding field values. Available in the `$setWindowFields` stage. *New in version 5.3.*
`$locf`	Last observation carried forward. Sets values for `null` and missing fields in a window to the last non-null value for the field. Available in the `$setWindowFields` stage. *New in version 5.2.*
`$max`	Returns the maximum value that results from applying an expression to each document. *Changed in version 5.0*: Available in the `$setWindowFields` stage.
`$min`	Returns the minimum value that results from applying an expression to each document. *Changed in version 5.0*: Available in the `$setWindowFields` stage.
`$minN`	Returns an aggregation of the `n` minimum valued elements in a group. Distinct from the `$minN` array operator. *New in version 5.2.* Available in `$group`, `$setWindowFields` and as an expression.
`$push`	Returns an array of values that result from applying an expression to each document. *Changed in version 5.0*: Available in the `$setWindowFields` stage.
`$rank`	Returns the document position (known as the rank) relative to other documents in the `$setWindowFields` stage partition. *New in version 5.0.*
`$shift`	Returns the value from an expression applied to a document in a specified position relative to the current document in the `$setWindowFields` stage partition. *New in version 5.0.*
`$stdDevPop`	Returns the population standard deviation that results from applying a numeric expression to each document. *Changed in version 5.0*: Available in the `$setWindowFields` stage.
`$stdDevSamp`	Returns the sample standard deviation that results from applying a numeric expression to each document. *Changed in version 5.0*: Available in the `$setWindowFields` stage.
`$sum`	Returns the sum that results from applying a numeric expression to each document. *Changed in version 5.0*: Available in the `$setWindowFields` stage.
`$top`	Returns the top element within a group according to the specified sort order. *New in version 5.2.* Available in the `$group` and `$setWindowFields` stages.
`$topN`	Returns an aggregation of the top `n` fields within a group, according to the specified sort order. *New in version 5.2.* Available in the `$group` and `$setWindowFields` stages.

For the pipeline stages, see Aggregation Stages.

Back
$vectorSearch

Next
$abs

© 2024 MongoDB, Inc.